

2026-04-15- 讨论

当前进展、问题收束 与下一步投稿判断

这次汇报只做三件事：给出现状判断、解释为什么先收束 project_1、以及明确需要拍板的问题。

project_1 优先级

pyfcstm vs
pyudbm

control-state 定义

博士研究进展对齐

project_1

建模主线 / 当前优先级最高

project_2

性质与场景生成 / 接口层

project_3

verification backend / 地基已起

project_4

iterative repair / 依赖前置成熟

日期 2026-04-15

明天这次讨论，我最希望先对齐三个决策

先拍板问题边界，再决定论文如何写和接下来 6 周怎么推

01 本学期主投稿是否明确锁定 `project\_1`

先做 focused paper，而不是一次性拉四个 project。

02 论文对象是否先限定为离散 `control-state layer`

把模式、阶段、互锁、恢复、局部定时这一层先讲透。

03 `pyfcstm` 与 `pyudbm` 是否分担建模基座与验证地基

一个回答目标对象，一个回答 timed backend。

后续展开顺序

1. 结论先给

2. project_1 文库证据

3. 基础设施与仓库进展

4. 拍板事项与倒排计划

一句话主线

不是再去找更多材料，而是让现有材料收敛成一篇问题边界清楚、对象清楚、基础设施落点清楚的会议论文。

当前最缺的不是材料，而是把问题对象收束清楚

一页结论先给出来，后面逐页用文库与仓库证据补强

1 先发 `project\_1`

本学期第一优先级应先把 focused paper 发出去。

2 真正缺的是收束

`project\_1` 当前不缺样本，缺的是问题定义。

3 先做离散 control-state

目标对象先落到模式、阶段、互锁、恢复、局部 timer。

4 `pyfcstm` 是研究答案

它应被写成 executable control-state infrastructure。

5 `project\_3` 有地基但未成型

`pyudbm` 已推进很深，但 verifyta 核心搜索仍缺。

6 反馈基础设施最关键

LLM-based modeling 的杀手锏是持续反馈，而不只是 prompt。

Take-home

先收束对象，再拉厚实验；只要定义稳住，现有文库和仓库推进已经足以支撑一篇 conference-style 论文。

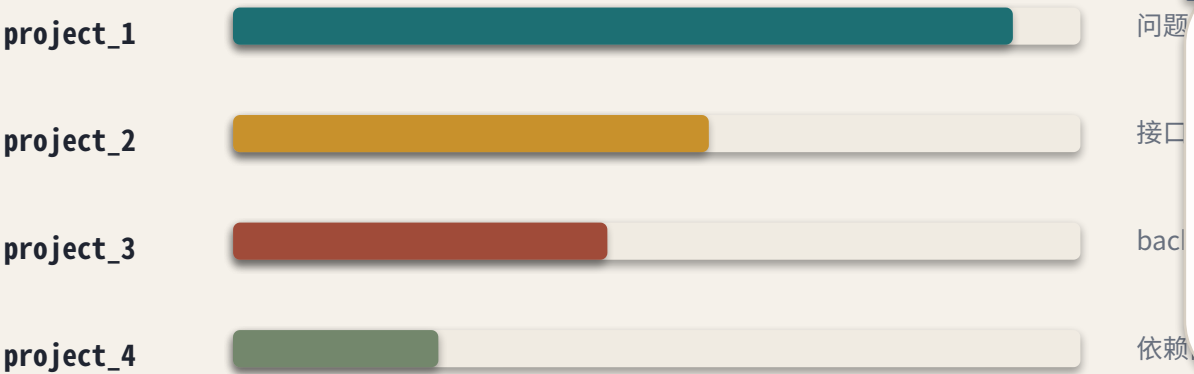
站在本学期投稿窗口看，只有 project_1 适合先冲出去

近端会议时间窗口 + 当前准备度 两条线必须同时看

prior-year deadline 参考



当前准备度



页面结论

RE / MoDELS 风格窗口基本已过。
ASE-style automation + infrastructure 写法仍是近端最现实目标。
所以这学期不宜再把主问题做散。

四个 project 是一条闭环，但近端主线必须先落在 project_1

先把建模对象和目标形式主义立住，后面的 scenario、verification、repair 才有共同基座



当前判断

博士主线仍然是生成 - 验证 - 修复闭环；只是本学期要先把建模对象与目标形式主义打稳，后续几项才有共同基座。

project_1 的说服力来自三条文库线加上一条基础设施线

不是平行摆设，而是共同回答“该建什么、凭什么、如何落地”



`baselines/` 解决的是“该和谁比，以及差距究竟在哪里”

这不是领域均匀采样，而是围绕 project_1 可比性刻意筛出的比较集

状态分布

62

baseline 条目

直接比较对象已经够用



绿色条目不多，但它们正是明天汇报该重点讲的几篇。

direct baseline

正面回答“自由文本 -> 状态机”

邻近任务

需求 -> 行为模型 / model checking 修复

方法启发

workflow、反馈闭环、工具链集成

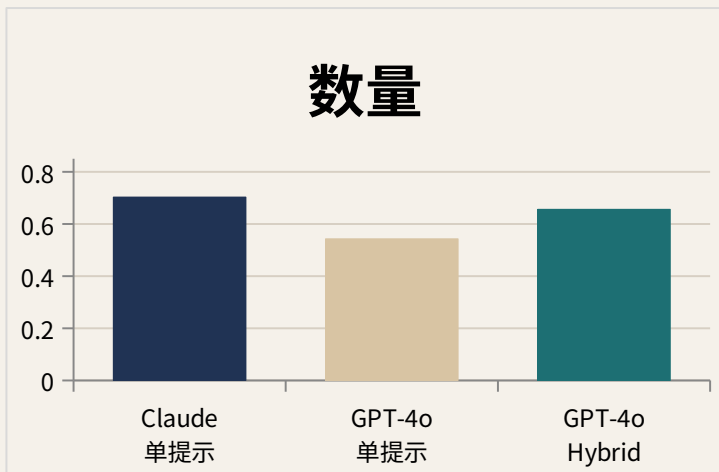
页面结论

baseline 文库已经够厚，关键是挑最硬的绿色条目来支撑主张。

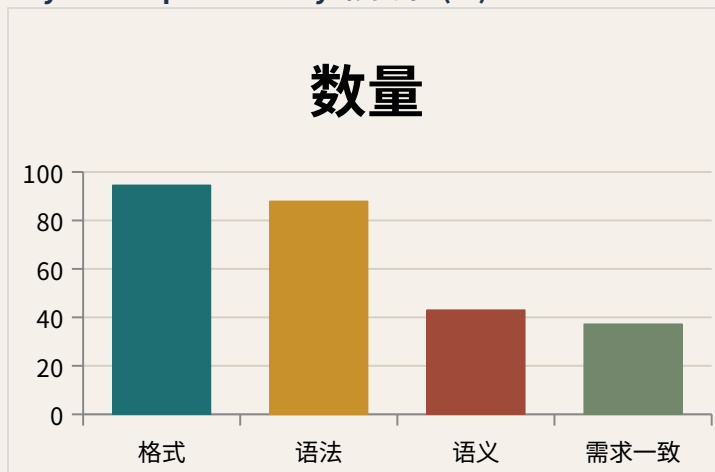
baseline 的共同趋势不是更花的 prompt，而是更强的工具反馈

几篇最值得讲的绿色条目给出的实证指向相当一致

2026 direct baseline F1



SysML empirical study 修复率 (%)



TTool AI

状态机评分 63 vs 58

速度快 15.2x

块图评分 81 vs 70

速度快 67.5x

IEC 61499 / workflow principles

一旦接上仿真、代码生成和 sanity checks，方法论就不再是一次 prompt。

Take-home

纯 prompt 可以给出骨架，但真正把质量拉上去的，是 model checking、仿真、编译检查和 workflow feedback。

`sources/` 不是自然分布样本，而是面向数据集建设的治理后主集

如果不先说明这一点，后面的统计就很容易被误读

前因

如果不先讲治理口径，后面所有 EFSM/HSM/T0/T1 的比例都会被误读成自然分布。

因此

所以这一页先解释筛选逻辑，再看统计。

Warning

后面所有 EFSM / HSM / T0 / T1 的比例，首先反映的是收录策略与数据集治理目标，其次才是领域文献现象本身。

治理流程

广撒网检索

先把真实控制系统状态逻辑的文献面铺开。

标准化治理

按论文级、案例级、主类型、时间级别等口径做筛。

主集保留

优先保留 EFSM/HSM、T0/T1、细节高、不强趋同的案例。

五套治理口径

治理目标

优先保留 `EFSM/HSM`。
优先保留 `T0/T1`。
优先保留细节充实、可转为可执行样本的案例。
对强趋同簇做降采样控制。

口径	作用
论文级可用性	判断单篇论文整体上能否形成可靠样本
案例级角色	区分核心保留、清洗后保留、降采样保留
细节充实度	判断是否足以支撑高质量建模与评测
主类型	回答“默认应把它理解成哪类状态机”
时间级别	回答时间语义到底只是局部 timer 还是强实时 / 连续耦合

`sources/` 的总体统计已经足以支撑数据集和问题边界判断

统计项必须带定义列，否则数字没有解释力

前因

样本主链已经够厚，不再是“先去找样本”的阶段。

因此

所以这页要把数量和定义一起说清楚。

787

论文总数

正式入账论文目录

715

论文级

可直接支撑可靠样本

746

正例案例

当前样本主链总量

685

核心保留

主数据集主链

指标	定义	数量
论文总数	当前 `sources/` 已正式入账的论文目录总数	787
论文级 ` `	单篇论文整体上可直接支持可靠样本抽取	715
论文级 ` `	单篇论文仍有价值，但需要额外整理或局部补证	16
论文级 ` `	单篇论文最终未形成可靠样本	56

类别	定义	数量
正例案例总数	正式入账、可作为控制状态样本讨论的案例条目	746
` ` 核心保留	细节和分布都足够好，适合进入主数据集主链	685
` ` 清洗后保留	有价值，但需要进一步清洗、补证或统一口径	20
` ` 降采样保留	本身可用，但因分布控制原因不应过量进入主集	41

Take-home

主链样本已经够厚；现在真正该做的是围绕它定义论文对象、baseline 口径和可执行实验。

主类型分布说明真实样本主链是 `FSM / EFSM / HSM`

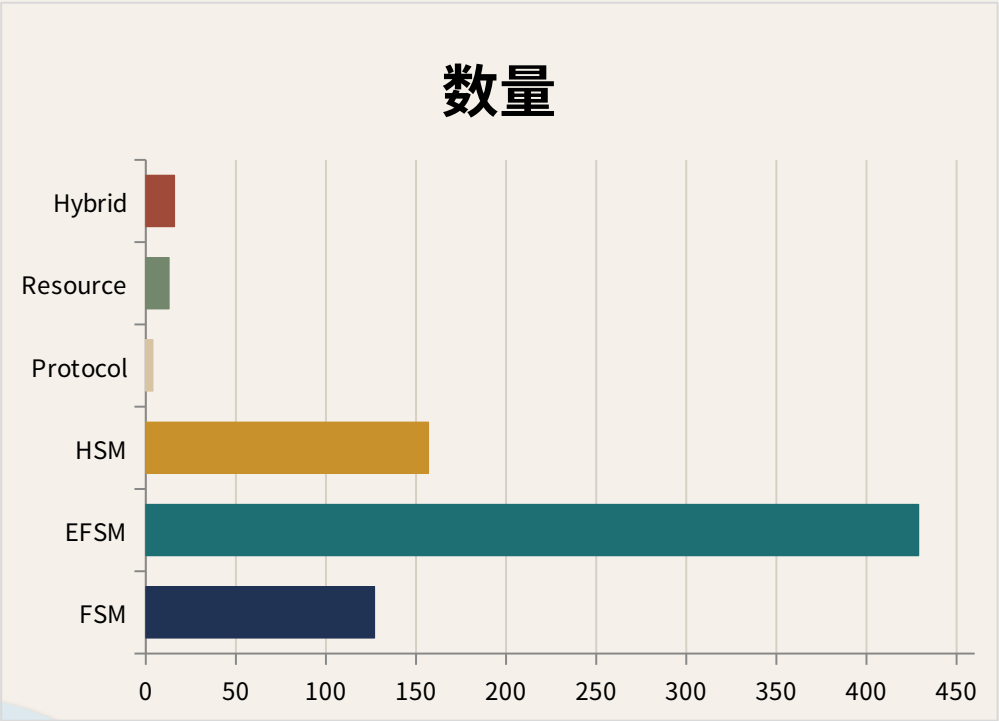
713 / 746 条正例都落在离散控制状态层附近

前因

主类型分布不只是数字，它直接决定论文对象该落在哪条主链上。

因此

所以这页要读成“control-state 占绝对主体”。



主类型	定义
FSM	普通离散阶段机，少量条件并入状态也不会明显失真
EFSM	离散状态仍是主体，但关键语义依赖 guard / effect 数据面
HSM	高层模式和低层子状态关系本身就是主要结构事实
Protocol	多角色请求、授权、接管等交互顺序是核心语义
Resource-flow	正确性主要由资源占用、互斥、释放关系决定
Hybrid	连续动力学或连续控制律是语义不可删的一部分

关键比例

713 / 746 = 95.6% 的主集正例都落在 `FSM + EFSM + HSM` 这条离散控制状态主链上。

时间级别与结构标签说明 “离散控制” 不等于 “扁平简单 FSM”

T0/T1 占主导，但显式时钟与层次结构都不是小噪声

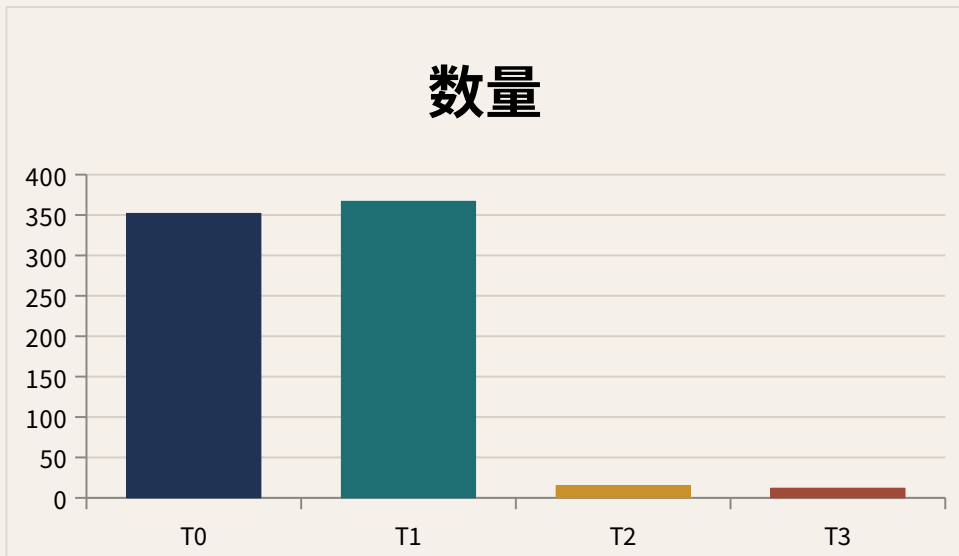
前因

离散主链并不意味着“模型可以极简”。

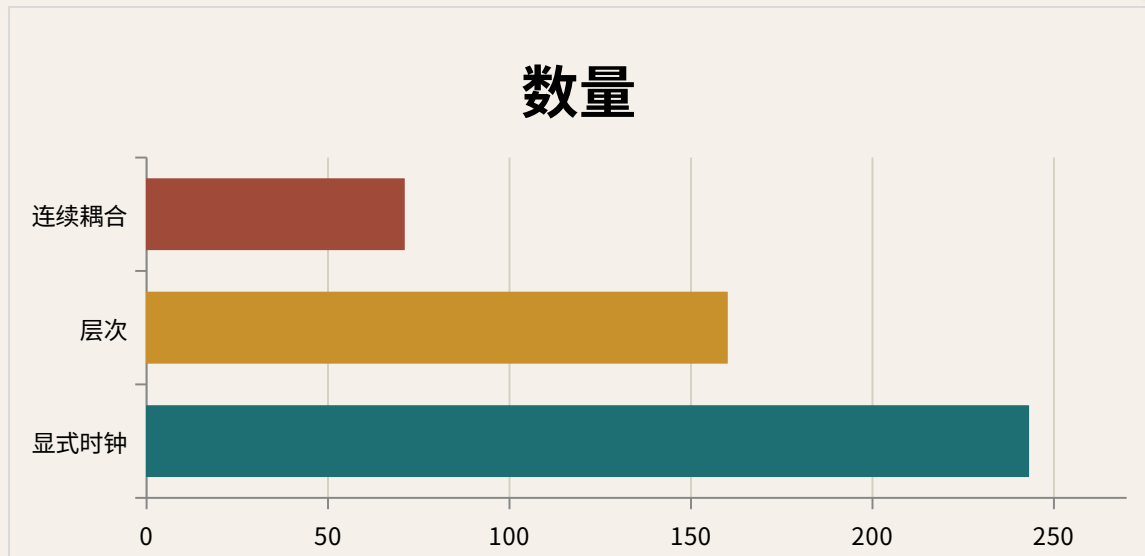
因此

所以 timer、显式时钟和 hierarchy 都必须被保留下来。

时间级别分布



结构标签覆盖



结论 1

‘T0 + T1 = 719 / 746’，短期最稳的对象仍是离散控制 + 局部工程定时。

结论 2

显式时钟与层次结构必须保留，所以目标形式主义不能退回最平的 FSM。

代表样本显示：真实控制系统里至少并存五类不同建模难点

同样都叫“状态机”，但它们的主难点并不相同

Framing

这些样本不是用来“凑例子”，而是用来证明“状态机”一词背后其实罩着不同问题，因此论文对象必须主动收束。

场景	标签	关键建模难点	对论文对象的启发
洗衣机 PLC	EFSM / T1	阶段链 + timer + guard	真实工业控制里“阶段 + timer”是高频对象
电梯 PLC	EFSM / T1	请求队列、方向优先、门控时长	顺序控制与门控规则构成主问题
铁路联锁	Resource / T1-T2	资源互斥、锁闭、释放	有些“状态机”本质上更像强 guard / 资源系统
UAV 分层任务	HSM / T0-T1	mission supervisor + 子层任务控制	层次任务控制是另一条高频主线
外骨骼步态控制	Hybrid / T3	连续相变量与模式切换强耦合	确实存在第一类连续 / 混成问题

结论

离散顺序控制、层次任务、资源互锁、连续耦合至少是四类不同问题。

`state\_machine\_types/` 证明“状态机”是一整个家族而不是单对象

问题不在于有没有状态机，而在于我们到底主动选择其中哪一支

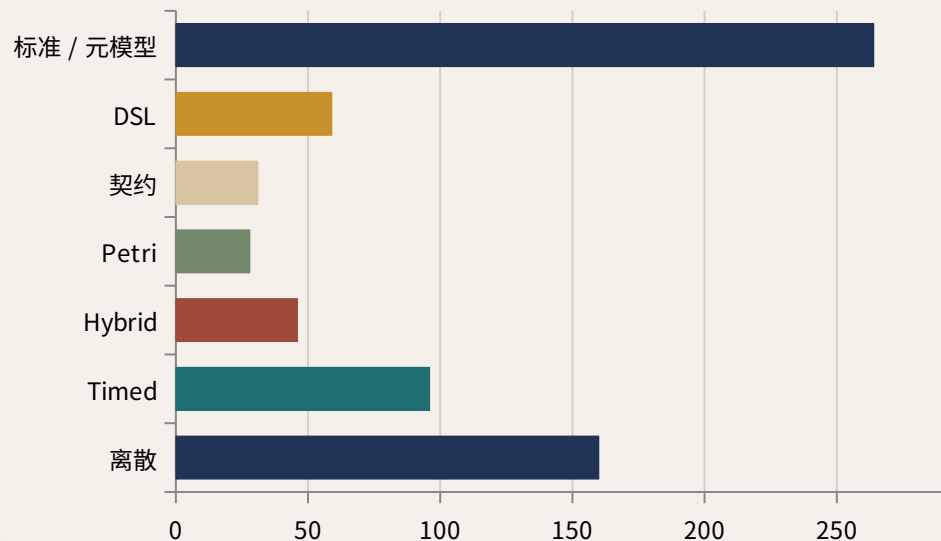
前因

既然“状态机”本来就是家族，project_1 就不能假装目标对象天然唯一。

因此

所以必须主动选 control-state + infrastructure 这条分支。

数量



状态机家族

control-state

FSM / EFSM / HSM

timed

TA / clocks

hybrid

continuous coupling

resource /

contract

infrastructure

DSL / 标准 / 元模型 / 执行载体

因此

对 project_1 来说，最自然的选择不是“任意状态机”，而是 control-state profile 加执行基础设施。

这篇论文更稳的对象应是控制系统的离散 control-state layer

离散监督 / 顺序控制 与 连续 / 混成控制 在建模视角上其实是两类问题

维度	离散监督 / 顺序控制 / 模式管理	连续 / 混成控制中的模式切换
典型系统	PLC 、电梯、铁路联锁、门控、任务控制、异常恢复链	ABS / BBW 、外骨骼步态、强时序医疗控制、动力学模式切换
状态的主要含义	模式、阶段、权限、流程位置、故障恢复位置	控制律区间、相变量阶段、连续动力学运行区间
核心建模难点	guard 、事件、互锁、局部 timer 、异常链	时钟组合、连续变量、动力学耦合、 hybrid semantics
更自然的模型	FSM / EFSM / HSM / control-state DSL	Timed / Hybrid 一类对象
更自然的反馈基础设施	parser 、 simulator 、结构校验、可执行 trace	timed/hybrid verification 、数值仿真、 symbolic reachability

论文对象建议
project_1 第一篇论文先解左列问题；右列问题不否认其重要性，但放到后续延展更稳。

`pyfcstm` 从 2 月底到现在已经具备可执行基础设施的骨架

它已经不是单文件 demo 级 DSL，而是在往工程可维护对象走

dcf1f70

main HEAD

2026-02-28

2026-04-14

import / 模块化

import 语法、模型组装、
目录入口、editor
support

执行语义

`if` 语句、运行时递归执
行、symbolic if-block
execution

代码生成

Python / C 模
板、`c\_poll` 路
线、template tests

工具支持

PlantUML 导出、文档、
教程、VS Code 支持

验证预备

solver / verify
groundwork、symbolic
witness 方向

页面结论

`pyfcstm` 现在最值得强调的不是 feature 数量，而是它已经形成了一条从模型表达、执行、代码生成到后续验证接口的连续能力带。

`pyfcstm` 在论文里应被写成目标形式主义与闭环基座

把“自然语言生成状态机”提升成“自然语言生成可执行形式模型”

target profile

不直接追求宽语义 UML/SCXML，而是主动收束成 control-state profile。

executable semantics

一生成就可解析、可执行、可继续进入 parser/runtime 反馈。

需求文本

LLM

`pyfcstm`
control-state DSL

parser /
runtime

simulation /
codegen /
verification
hooks

formal core

形式化核心与外部 action 显式隔离，方便验证与后续修复。

cross-project base

它为 project_2/3/4 提供共同模型对象，而不是一次性 demo。

结论

`pyfcstm` 回答的是“LLM 到底该生成什么对象，为什么这个对象既可执行又适合闭环”这个研究问题，而不是简单实现问题。

`project\_3` 的实质推进主要在 `pyudbm`，但核心搜索仍缺

backend 不是空白，只是还没长成第一版 profile-guided verifier

前因

project_3 很容易被看成“目录还空，所以几乎没动”。

因此

所以这页要把“地基已做很多”和“完整 verifier 仍未成型”分开说。

a8d0649

main HEAD

本地 clone 于 `~/oo-projects/pyudbm`

符号核 | UDBM API 与 UCDD 路线已立住

模型前端 | UTAP 绑定、`load\_xml`、`parse\_query` 已有

query + corpus | roundtrip 与 178 个官方样本已成体系

文献与路线 | TA / UPPAAL 阅读地图与 roadmap 持续维护

层	已有	仍缺 / 影响
基础符号层	已有	不是主要瓶颈
模型 / query 接口	已有	官方样本和 roundtrip 已成体系
symbolic reachability	仍缺	无法形成真正 verifier
$A[] / E<>$ 求值	仍缺	性质检查闭环还接不上
witness / counterexample	仍缺	反馈链条不完整
verifyta 核心搜索	仍缺	这才是 project_3 真正未过的门槛

页面结论

`project\_3` 现在最合理的定位，是继续让 `pyudbm` 沉淀 timed backend，而不是抢在 `project\_1` 之前承担主投稿。

我越来越相信基础设施反馈才是 LLM-based modeling 的真正杀手锏

parser、runtime、simulation、model checking 和 regression traceability 会把问题性质彻底改变

前因

单次生成很快会碰到 action、语义一致性和可执行性瓶颈。

因此

所以真正把质量拉高的，不是 prompt 花样，而是反馈基础设施。

2026 direct baseline

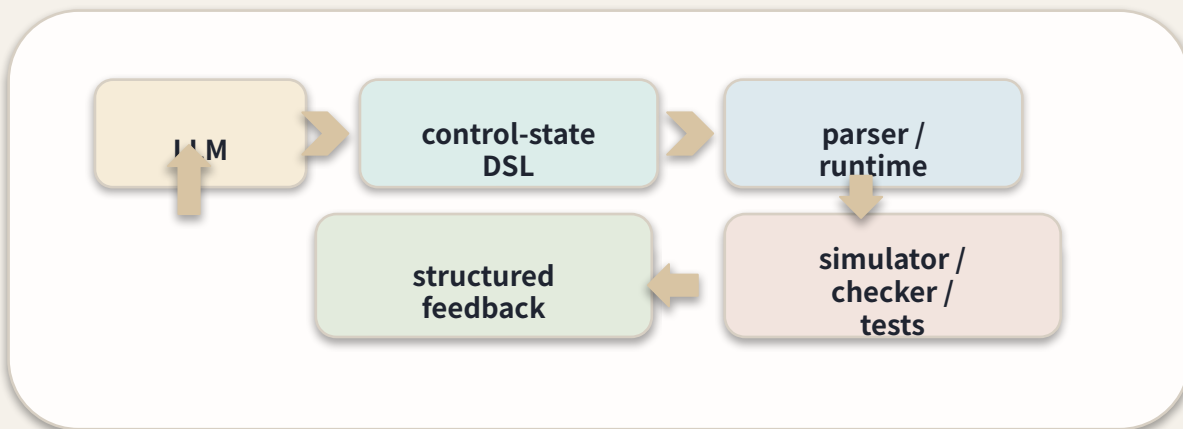
0.7029 vs 0.5431

Hybrid 流程能明显拉回弱模型

SysML empirical

94.6 / 88.0 / 43.1 / 37.3

规则反馈有效，但还不够



TTool AI

63 vs 58 , 15.2x

81 vs 70 , 67.5x

workflow principles

可信 GenAI 依赖
decomposition、sanity
checks、traceability

Take-home

与其继续拼 prompt，不如把模型输出放进能持续打分、回传反例、约束格式和校验语义的环境里。

如果明天拍板这五件事，后续 6 周的推进路径会更稳

收束问题对象、拉厚实验、准备 conference-style 论文

6 周倒排

第 1-2 周

收束问题定义与论文主张

第 3-4 周

固化样本与 baseline 实验

第 5-6 周

完成初稿与 venue 策略

决策 1

本学期主投稿是否锁定 `project\_1`

决策 2

论文对象是否先限定为离散
`control-state layer`

决策 3

`pyfcstm` 是否正面写成 target
formalism / executable IR

决策 4

`pyudbm` 是否继续做 backend 地基而不抢主线

决策 5

近端写法是否按 ASE-style
automation + feedback
infrastructure 组织

Closing

如果这五件事里有三四件能拍板，后面 6 周就可以沿着“对象收束 -> 实验拉厚 -> 初稿成型”稳步推进。